

**“UNIVERSIDAD CENTROAMERICANA
JOSÉ SIMEÓN CAÑAS”
DEPARTAMENTO DE ELECTRÓNICA Y INFORMÁTICA**



BASES DE DATOS

PROYECTO FINAL

CATEDRÁTICO:

ING. JAMES EDWARD HUMBERSTONE MORALES

ESTUDIANTES:

BARRERA GOMEZ, HERALDO RIQUELMY

ALVAREZ MORALES, JOSÉ ARIEL

FUENTES SANDOVAL, EMELY RUBÍ

AYALA JANDRES, DIANA REBECA

CLAROS LÓPEZ, ERICK JOSE

LEMUS SIBRIAN, ISRAEL

SECCIÓN: 02

CICLO 01-26

FECHA DE ENTREGA: VIERNES 19 DE JUNIO 2026

Documentación Técnica del Diseño e Implementación del Proyecto

Se analizaron las necesidades de un sistema de gestión hotelera para administrar hoteles, habitaciones, huéspedes, reservaciones, estancias, servicios adicionales, facturación y empleados. A partir de las reglas de negocio se identificaron las entidades, atributos y relaciones necesarias para el sistema.

Reglas de negocio — Sistema de Reservas de Hotel

1. Un hotel administra sus habitaciones, huéspedes, reservaciones, servicios adicionales y facturación de estancias.
2. Un hotel puede tener varias habitaciones registradas.
3. Cada habitación pertenece a un tipo de habitación.
4. Un huésped puede realizar múltiples reservaciones.
5. Una reservación pertenece a un solo huésped.
6. Una reservación cubre una habitación específica durante un rango de fechas.
7. Una habitación no puede estar reservada dos veces en el mismo período.
8. Durante la estancia, el huésped puede consumir servicios adicionales como restaurante, lavandería o spa.
9. Cada consumo de servicio debe estar asociado a una reservación.
10. Al realizar el check-out se genera una factura consolidada.
11. La factura debe incluir el costo de la habitación y los servicios adicionales consumidos.
12. Un empleado puede registrar reservaciones, gestionar el proceso de estancia del huésped (check-in y check-out) y generar las facturas correspondientes.
13. Una reservación puede tener estados como pendiente, completada, cancelada o finalizada.
14. Una habitación puede tener estados como disponible, ocupada o en mantenimiento.
15. Una reservación debe tener registrada una fecha de entrada y una fecha de salida válidas.

16. Una factura puede contener uno o varios detalles de factura, donde se especifican los conceptos cobrados, cantidades, valores unitarios y subtotales.

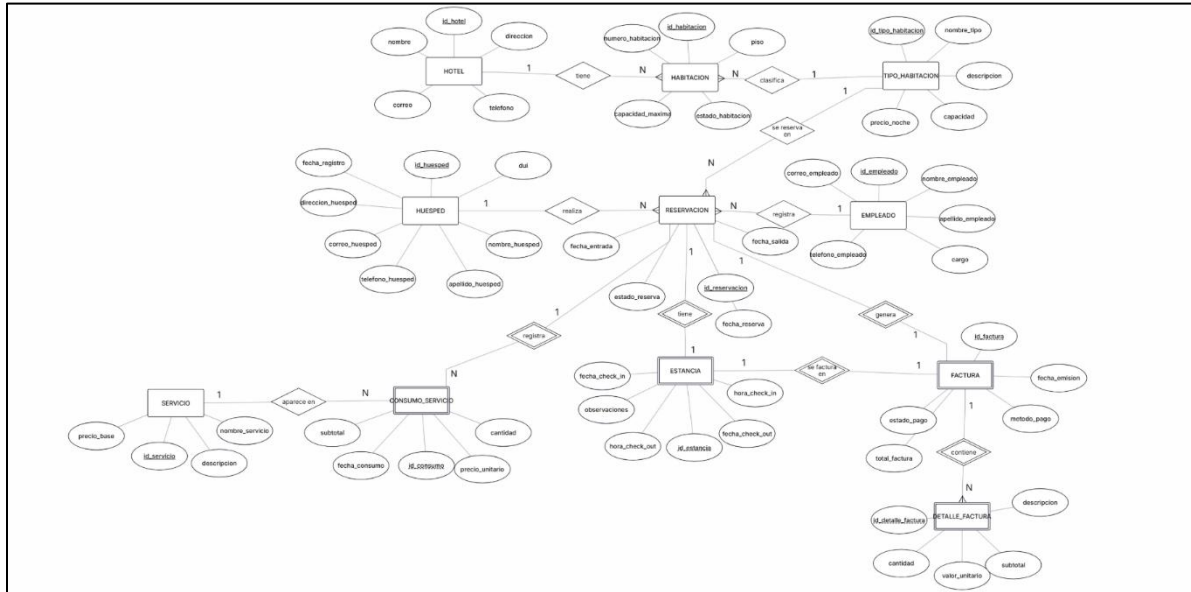
Diseño Conceptual

Se elaboró un diagrama entidad-relación (ER) utilizando el modelo de Chen para representar gráficamente las entidades del sistema, sus atributos, claves primarias y las relaciones existentes entre ellas.

Las entidades principales identificadas fueron:

- hotel
- habitacion
- tipo_habitacion
- huesped
- reservacion
- estancia
- servicio
- consumo_servicio
- factura
- detalle_factura
- empleado

Diagrama Entidad-Relación en notación Chen



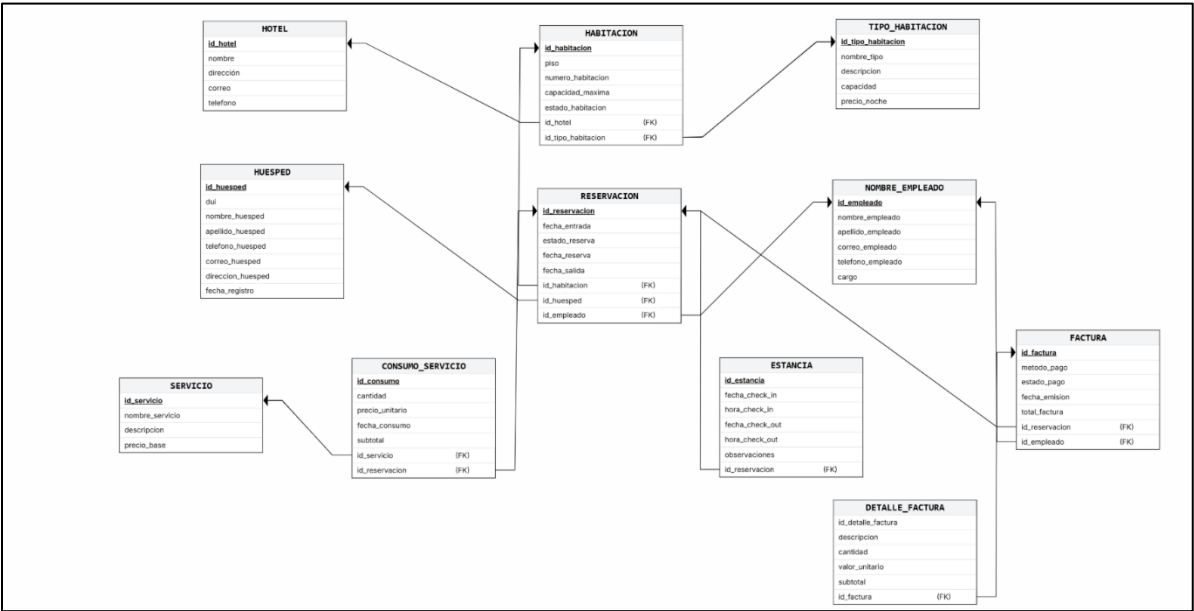
Diseño Lógico

A partir del modelo entidad-relación se construyó el modelo relacional (MR), transformando cada entidad en una tabla e identificando las claves primarias y foráneas necesarias para mantener la integridad referencial.

Se definieron relaciones de:

- Uno a muchos (1:N)
- Uno a uno (1:1)
- Dependencias entre entidades fuertes y débiles

Esquema relacional normalizado (3FN)



Diccionario de datos

Tabla: hotel

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_hotel	bigint	8 bytes	PK, identity, not null	Identificador único del hotel.
nombre	varchar	100	not null	Nombre del hotel.
direccion	varchar	200	not null	Dirección física del hotel.
correo	varchar	150	unique, not null, check	Correo electrónico del hotel.
telefono	varchar	20	not null	Número telefónico del hotel.

Tabla: tipo_habitacion

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_tipo_habitacion	bigint	8 bytes	PK, identity, not null	Identificador único del tipo de habitación.
nombre_tipo	varchar	80	unique, not null	Nombre del tipo de habitación.
descripcion	varchar	300	null	Descripción del tipo de habitación.
capacidad	int	4 bytes	not null, check	Capacidad máxima del tipo de habitación.
precio_noche	numeric	10,2	not null, check	Precio por noche del tipo de habitación.

Tabla: habitacion

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_habitacion	bigint	8 bytes	PK, identity, not null	Identificador único de la habitación.
piso	int	4 bytes	not null, check	Piso donde se encuentra la habitación.
numero_habitacion	varchar	10	not null, unique compuesto	Número asignado a la habitación.
capacidad_maxima	int	4 bytes	not null, check	Capacidad máxima de huéspedes permitida.
estado_habitacion	varchar	20	not null, default, check	Estado actual de la habitación.
id_hotel	bigint	8 bytes	FK, not null	Hotel al que pertenece la habitación.
id_tipo_habitacion	bigint	8 bytes	FK, not null	Tipo de habitación asignado.

Tabla: nombre_empleado

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_empleado	bigint	8 bytes	PK, identity, not null	Identificador único del empleado.
nombre_empleado	varchar	100	not null	Nombre del empleado.
apellido_empleado	varchar	100	not null	Apellido del empleado.
correo_empleado	varchar	150	unique, not null, check	Correo electrónico del empleado.
telefono_empleado	varchar	20	null	Número telefónico del empleado.
cargo	varchar	80	not null	Cargo o puesto del empleado.

Tabla: huesped

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_huesped	bigint	8 bytes	PK, identity, not null	Identificador único del huésped.
dui	varchar	15	unique, not null	Documento único de identidad del huésped.
nombre_huesped	varchar	100	not null	Nombre del huésped.
apellido_huesped	varchar	100	not null	Apellido del huésped.
telefono_huesped	varchar	20	null	Teléfono del huésped.
correo_huesped	varchar	150	unique, not null, check	Correo electrónico del huésped.
direccion_huesped	varchar	200	null	Dirección del huésped.
fecha_registro	date	4 bytes	not null, default	Fecha en que el huésped fue registrado.

Tabla: reservacion

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_reservacion	bigint	8 bytes	PK, identity, not null	Identificador único de la reservación.
fecha_entrada	date	4 bytes	not null, check	Fecha programada de entrada.
fecha_salida	date	4 bytes	not null, check	Fecha programada de salida.
fecha_reserva	timestamp	8 bytes	not null, default	Fecha y hora en que se registró la reservación.
estado_reserva	varchar	20	not null, default, check	Estado actual de la reservación.
id_habitacion	bigint	8 bytes	FK, not null	Habitación asignada a la reservación.
id_huesped	bigint	8 bytes	FK, not null	Huésped que realiza la reservación.
id_empleado	bigint	8 bytes	FK, not null	Empleado que registra la reservación.

Tabla: estancia

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_estancia	bigint	8 bytes	PK, identity, not null	Identificador único de la estancia.
fecha_check_in	date	4 bytes	not null	Fecha real de ingreso del huésped.
hora_check_in	time	8 bytes	not null	Hora real de ingreso del huésped.
fecha_check_out	date	4 bytes	null, check	Fecha real de salida del huésped.
hora_check_out	time	8 bytes	null	Hora real de salida del huésped.
observaciones	text	variable	null	Observaciones relacionadas con la estancia.
id_reservacion	bigint	8 bytes	FK, unique, not null	Reservación asociada a la estancia.

Tabla: servicio

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_servicio	bigint	8 bytes	PK, identity, not null	Identificador único del servicio.
nombre_servicio	varchar	100	unique, not null	Nombre del servicio ofrecido.
descripcion	varchar	300	null	Descripción del servicio.
precio_base	numeric	10,2	not null, check	Precio base del servicio.

Tabla: consumo_servicio

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_consumo	bigint	8 bytes	PK, identity, not null	Identificador único del consumo de servicio.
cantidad	int	4 bytes	not null, check	Cantidad consumida del servicio.
precio_unitario	numeric	10,2	not null, check	Precio unitario aplicado al consumo.
fecha_consumo	timestamp	8 bytes	not null, default	Fecha y hora en que se realizó el consumo.
subtotal	numeric	10,2	generado, stored	Subtotal calculado automáticamente.
id_servicio	bigint	8 bytes	FK, not null	Servicio consumido.
id_reservacion	bigint	8 bytes	FK, not null	Reservación asociada al consumo.

Tabla: factura

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_factura	bigint	8 bytes	PK, identity, not null	Identificador único de la factura.
metodo_pago	varchar	30	not null, check	Método de pago utilizado.
estado_pago	varchar	20	not null, default, check	Estado actual del pago.
fecha_emision	timestamp	8 bytes	not null, default	Fecha y hora en que se emitió la factura.
total_factura	numeric	10,2	not null, default, check	Monto total de la factura.
id_reservacion	bigint	8 bytes	FK, unique, not null	Reservación asociada a la factura.
id_empleado	bigint	8 bytes	FK, not null	Empleado que genera la factura.

Tabla: detalle_factura

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_detalle_factura	bigint	8 bytes	PK, identity, not null	Identificador único del detalle de factura.
descripcion	varchar	200	not null	Concepto cobrado en la factura.
cantidad	int	4 bytes	not null, check	Cantidad del concepto facturado.
valor_unitario	numeric	10,2	not null, check	Valor unitario del concepto facturado.
subtotal	numeric	10,2	generado, stored	Subtotal calculado automáticamente.
id_factura	bigint	8 bytes	FK, not null	Factura a la que pertenece el detalle.

Tabla: auditoria_precio_tipo_habitacion

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_auditoria	bigint	8 bytes	PK, identity, not null	Identificador único del registro de auditoría.
id_tipo_habitacion	bigint	8 bytes	not null	Tipo de habitación cuyo precio fue modificado.
precio_anterior	numeric	10,2	null	Precio anterior del tipo de habitación.
precio_nuevo	numeric	10,2	null	Precio nuevo del tipo de habitación.
fecha_cambio	timestamp	8 bytes	default	Fecha y hora del cambio realizado.
usuario_bd	text	variable	default	Usuario de base de datos que realizó el cambio.

Tabla: auditoria_reservacion

Nombre del Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
id_auditoria	bigint	8 bytes	PK, identity, not null	Identificador único del registro de auditoría.
operacion	text	variable	not null	Tipo de operación realizada: INSERT, UPDATE o DELETE.
id_reservacion	bigint	8 bytes	null	Reservación afectada por la operación.
datos_antes	jsonb	variable	null	Datos anteriores al cambio realizado.
datos_despues	jsonb	variable	null	Datos posteriores al cambio realizado.
fecha_cambio	timestamp	8 bytes	default	Fecha y hora del cambio realizado.
usuario_bd	text	variable	default	Usuario de base de datos que realizó la operación.

Implementación del Proyecto

Sentencias DDL

La implementación se realizó en PostgreSQL utilizando sentencias DDL para la creación de tablas, restricciones y relaciones.

Se utilizaron:

- primary key
- foreign key
- unique
- check
- generated always as identity
- valores por defecto (default)

Estas restricciones garantizan la integridad de los datos almacenados.

Población de datos

Se desarrolló un script DML para insertar datos de prueba en todas las tablas del sistema.

La información incluye:

- tipos de habitación
- habitaciones
- huéspedes
- empleados
- servicios
- reservaciones
- estancias
- facturas

- detalles de factura

Los datos fueron diseñados para simular escenarios reales de operación hotelera.

Consultas SQL

Con el objetivo de obtener información útil para la administración del hotel, se desarrollaron consultas SQL básicas y avanzadas que permiten analizar reservaciones, huéspedes, habitaciones, servicios y facturación.

Las consultas implementadas fueron las siguientes:

Consulta 1: Mostrar habitaciones registradas

Objetivo:

Mostrar las habitaciones registradas en el sistema, incluyendo su número, piso, capacidad máxima y estado actual.

Tablas utilizadas:

habitacion

Tipo de consulta:

Consulta básica con select y order by.

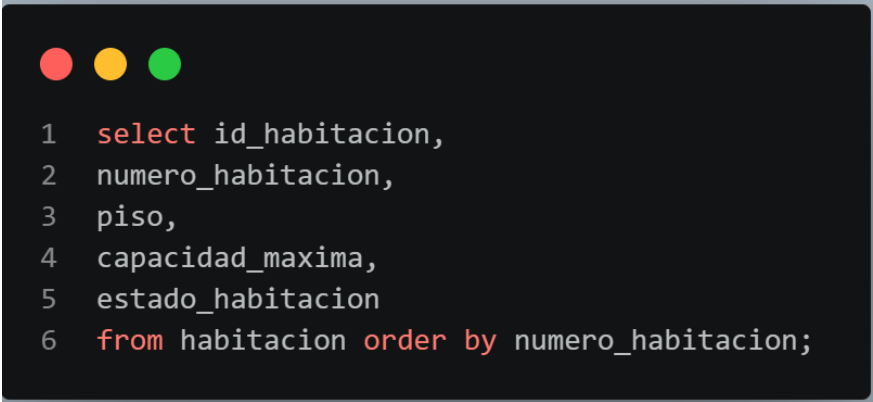
Descripción:

Esta consulta permite visualizar de forma ordenada todas las habitaciones registradas en el hotel. Es útil para que el personal administrativo pueda revisar la disponibilidad y el estado general de las habitaciones.

Resultado esperado:

Listado de habitaciones ordenadas por número de habitación.

Imagen de Referencia:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It displays a SQL query with line numbers 1 through 6.

```
1  select id_habitacion,  
2  numero_habitacion,  
3  piso,  
4  capacidad_maxima,  
5  estado_habitacion  
6  from habitacion order by numero_habitacion;
```

Consulta 2: Reservación, huésped, habitación y empleado

Objetivo:

Mostrar la información completa de cada reservación, relacionando el huésped, la habitación asignada y el empleado que realizó el registro.

Tablas utilizadas:

reservacion, huesped, habitacion, nombre_empleado

Tipo de consulta:

Consulta avanzada con múltiples inner join.

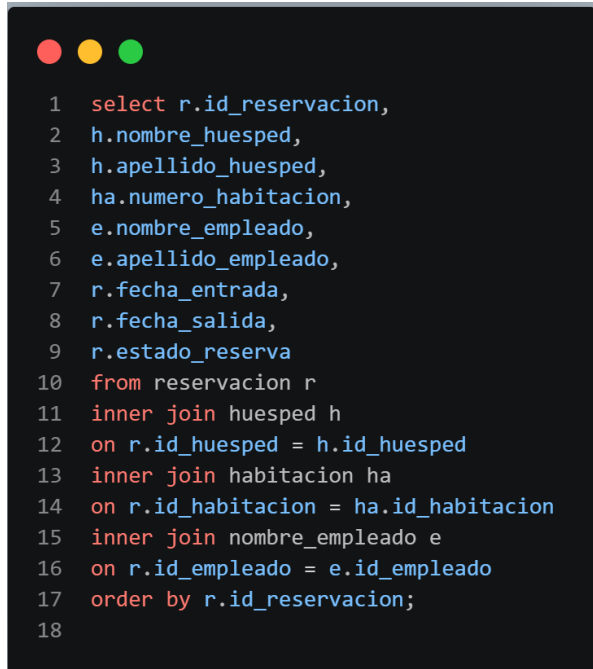
Descripción:

Esta consulta permite consultar el detalle de las reservaciones registradas en el sistema. Muestra quién realizó la reservación, qué habitación fue asignada, qué empleado la registró y las fechas de entrada y salida.

Resultado esperado:

Listado detallado de reservaciones ordenado por identificador de reservación.

Imagen de referencia:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The editor contains a SQL query with line numbers 1 through 18 on the left margin. The query is a complex JOIN statement selecting various fields from three tables: reservacion, huesped, and habitacion, and joining them with empleados. The query is as follows:

```
1  select r.id_reservacion,
2  h.nombre_huesped,
3  h.apellido_huesped,
4  ha.numero_habitacion,
5  e.nombre_empleado,
6  e.apellido_empleado,
7  r.fecha_entrada,
8  r.fecha_salida,
9  r.estado_reserva
10 from reservacion r
11 inner join huesped h
12 on r.id_huesped = h.id_huesped
13 inner join habitacion ha
14 on r.id_habitacion = ha.id_habitacion
15 inner join nombre_empleado e
16 on r.id_empleado = e.id_empleado
17 order by r.id_reservacion;
18
```

Consulta 3: Huéspedes con más de una reservación

Objetivo:

Identificar los huéspedes que han realizado más de una reservación en el hotel.

Tablas utilizadas:

huesped, reservacion

Tipo de consulta:

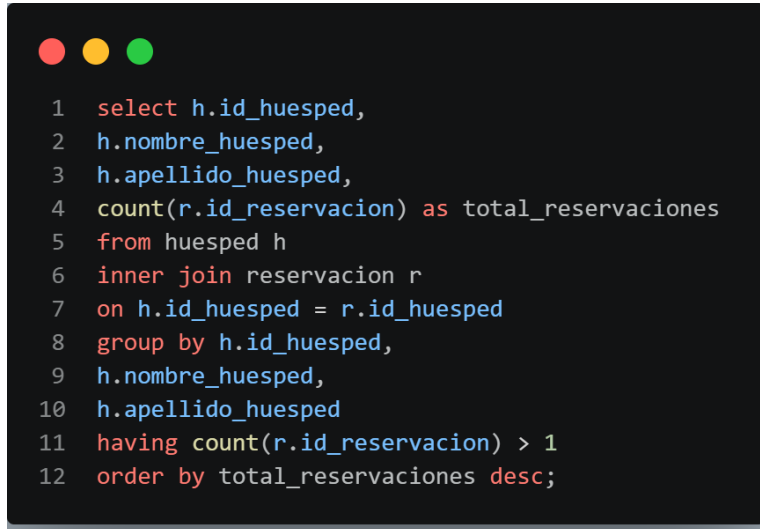
Consulta avanzada con inner join, group by, count y having.

Descripción:

Esta consulta agrupa las reservaciones por huésped y cuenta cuántas reservaciones ha realizado cada uno. El filtro having permite mostrar únicamente aquellos huéspedes con más de una reservación.

Resultado esperado:

Listado de huéspedes frecuentes ordenados por la cantidad de reservaciones realizadas.

Imagen de referencia:A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The editor contains a SQL query with line numbers 1 through 12. The query is a SELECT statement that joins the 'huesped' and 'reservacion' tables, groups the results by guest information, filters for guests with more than one reservation, and orders them by the total number of reservations in descending order.

```
1  select h.id_huesped,  
2  h.nombre_huesped,  
3  h.apellido_huesped,  
4  count(r.id_reservacion) as total_reservaciones  
5  from huesped h  
6  inner join reservacion r  
7  on h.id_huesped = r.id_huesped  
8  group by h.id_huesped,  
9  h.nombre_huesped,  
10 h.apellido_huesped  
11 having count(r.id_reservacion) > 1  
12 order by total_reservaciones desc;
```

Consulta 4: Habitaciones sin reservaciones**Objetivo:**

Identificar las habitaciones que no tienen reservaciones registradas.

Tablas utilizadas:

habitacion, reservacion

Tipo de consulta:

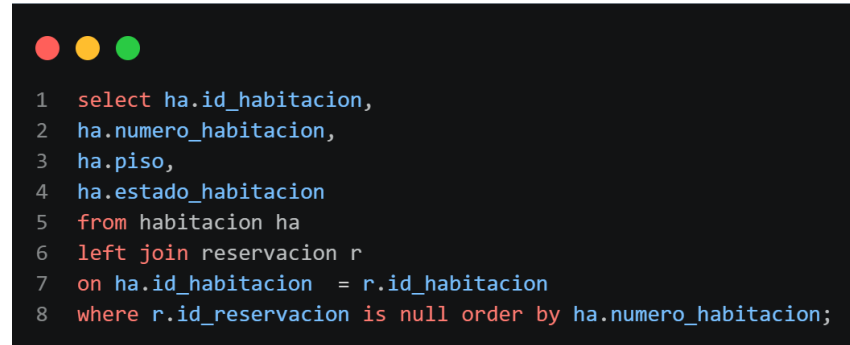
Consulta avanzada con left join e identificación de valores nulos.

Descripción:

Esta consulta muestra las habitaciones que no se encuentran asociadas a ninguna reservación. Es útil para detectar habitaciones que no han sido utilizadas o que podrían estar disponibles para futuras reservas.

Resultado esperado:

Listado de habitaciones sin reservaciones asociadas.

Imagen de referencia:A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The editor contains a SQL query with line numbers 1 through 8 on the left margin. The query is a SELECT statement that retrieves room details from a table named 'habitacion' (aliased as 'ha') using a LEFT JOIN with a table named 'reservacion' (aliased as 'r'). The join condition is 'ha.id_habitacion = r.id_habitacion'. The WHERE clause filters for reservations that are null ('r.id_reservacion is null') and orders the results by room number ('ha.numero_habitacion').

```
1 select ha.id_habitacion,  
2 ha.numero_habitacion,  
3 ha.piso,  
4 ha.estado_habitacion  
5 from habitacion ha  
6 left join reservacion r  
7 on ha.id_habitacion = r.id_habitacion  
8 where r.id_reservacion is null order by ha.numero_habitacion;
```

Consulta 5: Huéspedes que han gastado más de \$300**Objetivo:**

Mostrar los huéspedes cuyo gasto total facturado supera los \$300.

Tablas utilizadas:

huesped, reservacion, factura

Tipo de consulta:

Consulta avanzada con inner join, sum, group by y having.

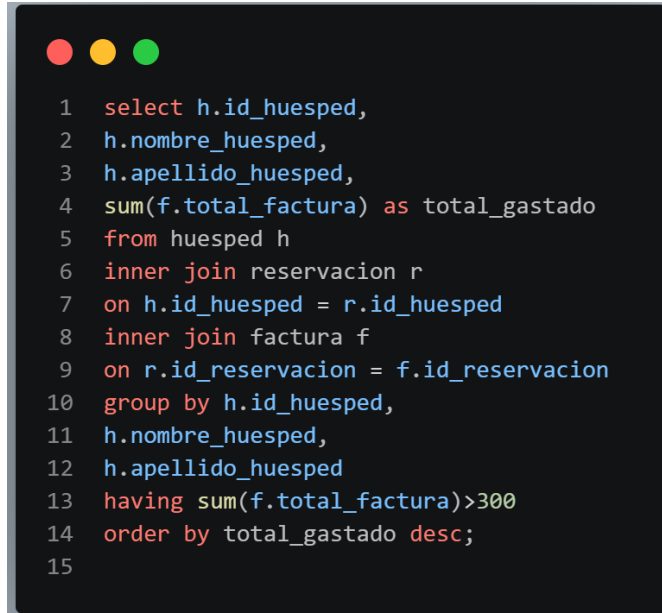
Descripción

Esta consulta calcula el total facturado por cada huésped y muestra únicamente aquellos cuyo gasto acumulado supera los \$300. Permite identificar clientes de alto valor para el hotel.

Resultado esperado:

Listado de huéspedes con mayor gasto, ordenado de mayor a menor total facturado.

Imagen de referencia:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The editor contains a SQL query with line numbers 1 through 15 on the left margin. The query is a SELECT statement that joins the 'huesped' table with 'reservacion' and 'factura' tables. It calculates the total amount spent for each guest and filters for those with a total greater than 300, ordered by total amount in descending order.

```
1  select h.id_huesped,  
2  h.nombre_huesped,  
3  h.apellido_huesped,  
4  sum(f.total_factura) as total_gastado  
5  from huesped h  
6  inner join reservacion r  
7  on h.id_huesped = r.id_huesped  
8  inner join factura f  
9  on r.id_reservacion = f.id_reservacion  
10 group by h.id_huesped,  
11 h.nombre_huesped,  
12 h.apellido_huesped  
13 having sum(f.total_factura)>300  
14 order by total_gastado desc;  
15
```

Consulta 6: Facturas superiores al promedio general

Objetivo:

Identificar las facturas cuyo monto total es mayor que el promedio general de facturación.

Tablas utilizadas:

reservacion, huesped, factura

Tipo de consulta:

Consulta avanzada con inner join y subconsulta.

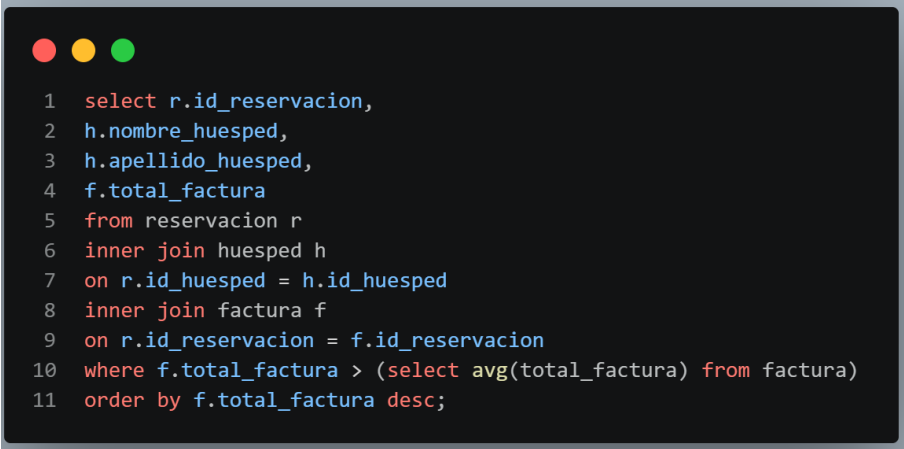
Descripción:

Esta consulta compara el total de cada factura contra el promedio general de todas las facturas registradas. Permite detectar reservaciones con montos superiores al promedio.

Resultado esperado:

Listado de facturas superiores al promedio, ordenadas de mayor a menor monto.

Imagen de referencia:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The editor contains a SQL query with line numbers 1 through 11. The query is a complex SELECT statement with multiple inner joins and a subquery in the WHERE clause.

```
1  select r.id_reservacion,  
2  h.nombre_huesped,  
3  h.apellido_huesped,  
4  f.total_factura  
5  from reservacion r  
6  inner join huesped h  
7  on r.id_huesped = h.id_huesped  
8  inner join factura f  
9  on r.id_reservacion = f.id_reservacion  
10 where f.total_factura > (select avg(total_factura) from factura)  
11 order by f.total_factura desc;
```

Consulta 7: Detalle completo de factura

Objetivo:

Mostrar el detalle completo de cada factura generada.

Tablas utilizadas:

factura, reservacion, huesped, detalle_factura

Tipo de consulta:

Consulta avanzada con múltiples inner join.

Descripción:

Esta consulta permite visualizar los conceptos cobrados en cada factura, incluyendo huésped, descripción del cargo, cantidad, valor unitario, subtotal, total de factura y método de pago.

Resultado esperado:

Listado detallado de facturas con sus respectivos conceptos cobrados.

Imagen de referencia:

```
1  select f.id_factura,  
2  h.nombre_huesped,  
3  h.apellido_huesped,  
4  df.descripcion,  
5  df.cantidad,  
6  df.valor_unitario,  
7  df.subtotal,  
8  f.total_factura,  
9  f.metodo_pago  
10 from factura f  
11 inner join reservacion r  
12 on f.id_reservacion = r.id_reservacion  
13 inner join huesped h  
14 on r.id_huesped = h.id_huesped  
15 inner join detalle_factura df  
16 on f.id_factura = df.id_factura  
17 order by f.id_factura,  
18 df.id_detalle_factura;
```

Consulta 8: Cantidad de reservaciones por estado

Objetivo:

Contabilizar cuántas reservaciones existen por cada estado.

Tablas utilizadas:

reservacion

Tipo de consulta:

Consulta con función de agregación count y group by.

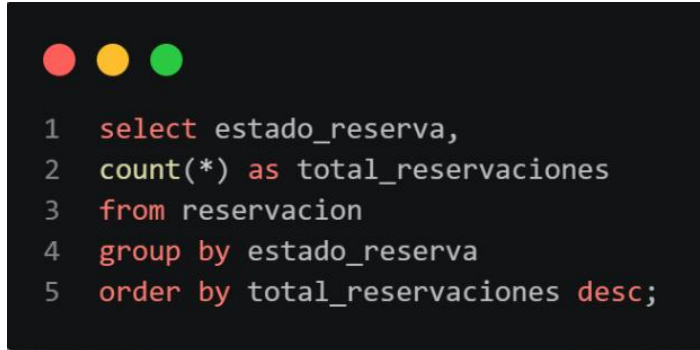
Descripción:

Esta consulta agrupa las reservaciones según su estado, permitiendo conocer cuántas están pendientes, confirmadas, canceladas o completadas.

Resultado esperado:

Cantidad total de reservaciones por estado.

Imagen de referencia:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It displays a SQL query with line numbers 1 through 5.

```
1  select estado_reserva,  
2  count(*) as total_reservaciones  
3  from reservacion  
4  group by estado_reserva  
5  order by total_reservaciones desc;
```

Consulta 9: Servicios más consumidos

Objetivo:

Identificar los servicios adicionales más consumidos por los huéspedes.

Tablas utilizadas:

servicio, consumo_servicio

Tipo de consulta:

Consulta avanzada con inner join, sum y group by.

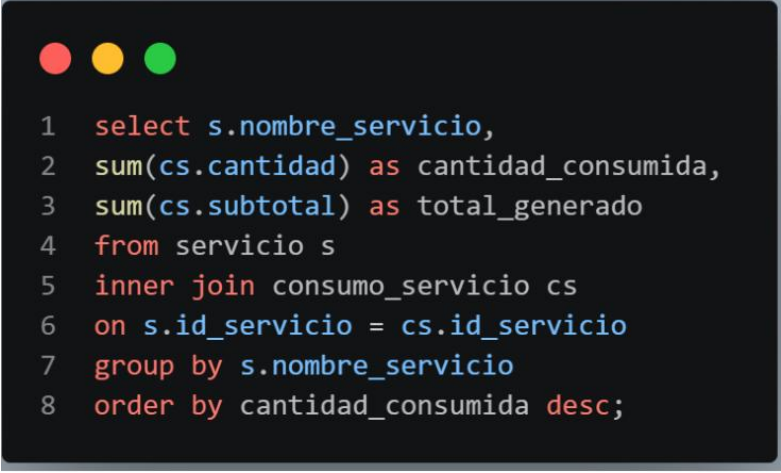
Descripción:

Esta consulta suma la cantidad consumida y el total generado por cada servicio adicional. Es útil para conocer qué servicios tienen mayor demanda dentro del hotel.

Resultado esperado:

Listado de servicios ordenados por cantidad consumida.

Imagen de referencia:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains an 8-line SQL query.

```
1  select s.nombre_servicio,  
2  sum(cs.cantidad) as cantidad_consumida,  
3  sum(cs.subtotal) as total_generado  
4  from servicio s  
5  inner join consumo_servicio cs  
6  on s.id_servicio = cs.id_servicio  
7  group by s.nombre_servicio  
8  order by cantidad_consumida desc;
```

Consulta 10: Ingresos mensuales

Objetivo:

Calcular los ingresos totales generados por mes.

Tablas utilizadas:

factura

Tipo de consulta:

Consulta con función de fecha extract, sum y group by.

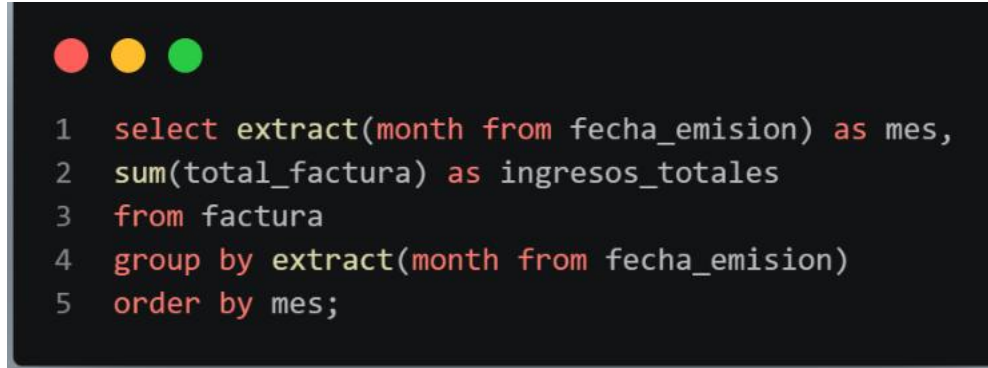
Descripción:

Esta consulta agrupa las facturas por mes de emisión y calcula el total de ingresos generados en cada período. Apoya el análisis financiero mensual del hotel.

Resultado esperado:

Listado de meses con su respectivo total de ingresos.

Imagen de referencia:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains a SQL query with line numbers 1 through 5.

```
1 select extract(month from fecha_emision) as mes,  
2 sum(total_factura) as ingresos_totales  
3 from factura  
4 group by extract(month from fecha_emision)  
5 order by mes;
```

Consulta 11: Promedio de gasto por huésped

Objetivo:

Calcular el gasto promedio realizado por cada huésped.

Tablas utilizadas:

huesped, reservacion, factura

Tipo de consulta:

Consulta avanzada con inner join, avg, round y group by.

Descripción:

Esta consulta obtiene el promedio de facturación por huésped, permitiendo analizar el comportamiento de consumo de los clientes registrados.

Resultado esperado:

Listado de huéspedes con su promedio de gasto, ordenado de mayor a menor.

Imagen de referencia:



```
1  select h.id_huesped,
2  h.nombre_huesped,
3  h.apellido_huesped,
4  round(avg(f.total_factura),2) as promedio_gastado
5  from huesped h
6  inner join reservacion r
7  on h.id_huesped = r.id_huesped
8  inner join factura f
9  on r.id_reservacion = f.id_reservacion
10 group by h.id_huesped,
11 h.nombre_huesped,
12 h.apellido_huesped
13 order by promedio_gastado desc;
```

Consulta 12: Duración de cada reservación

Objetivo:

Calcular la duración en días de cada reservación.

Tablas utilizadas:

reservacion

Tipo de consulta:

Consulta con operación de fechas y order by.

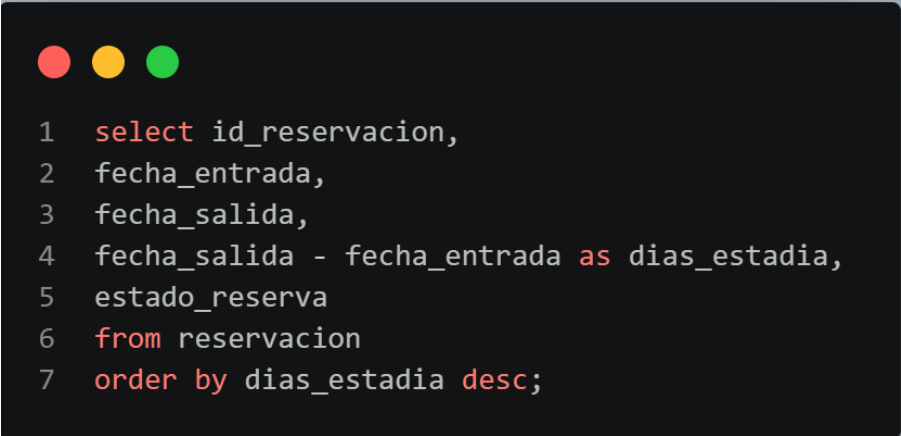
Descripción:

Esta consulta calcula la diferencia entre la fecha de salida y la fecha de entrada para determinar la cantidad de días de estancia de cada reservación.

Resultado esperado:

Listado de reservaciones con sus fechas y duración en días.

Imagen de referencia:



```
1  select id_reservacion,  
2  fecha_entrada,  
3  fecha_salida,  
4  fecha_salida - fecha_entrada as dias_estadia,  
5  estado_reserva  
6  from reservacion  
7  order by dias_estadia desc;
```

Estas consultas permiten transformar los datos almacenados en información útil para apoyar la gestión operativa y administrativa del hotel.

Transacciones

Transacción 1: Registrar nueva reservación

Objetivo:

Registrar una nueva reservación para un huésped, asignando una habitación disponible y el empleado responsable del registro.

Operaciones realizadas:

- Inserción de una nueva reservación.
- Registro de fechas de entrada y salida.
- Asociación de huésped, habitación y empleado.

Comandos utilizados:

- BEGIN
- INSERT
- COMMIT

Resultado esperado:

La reservación queda almacenada permanentemente en la base de datos una vez ejecutado el COMMIT.

Transacción 2: Registrar check-in

Objetivo:

Registrar el ingreso de un huésped al hotel mediante la creación de una estancia asociada a una reservación existente.

Operaciones realizadas:

- Inserción de un nuevo registro en la tabla estancia.
- Registro de fecha y hora de ingreso.
- Asociación con una reservación existente.

Comandos utilizados

- BEGIN
- INSERT
- COMMIT

Resultado esperado:

La estancia queda registrada correctamente y vinculada a la reservación correspondiente.

Transacción 3: Registrar consumo de servicio

Objetivo:

Registrar el consumo de un servicio adicional por parte de un huésped durante su estancia.

Operaciones realizadas:

- Inserción de un nuevo consumo de servicio.
- Registro de cantidad consumida.
- Asociación con un servicio y una reservación.

Comandos utilizados:

- BEGIN
- INSERT
- COMMIT

Resultado esperado:

El consumo queda almacenado en la base de datos y puede ser considerado posteriormente en la facturación.

Transacción 4: Demostración de ROLLBACK**Objetivo:**

Demostrar el funcionamiento del comando ROLLBACK para revertir cambios realizados dentro de una transacción.

Operaciones realizadas:

- Actualización temporal del estado de una habitación.
- Cancelación completa de los cambios mediante ROLLBACK.

Comandos utilizados:

- BEGIN
- UPDATE
- ROLLBACK

Resultado esperado:

La habitación conserva su estado original, ya que la modificación realizada no se guarda en la base de datos.

Propiedad ACID demostrada:

Atomicidad.

Transacción 5: Demostración de SAVEPOINT**Objetivo:**

Demostrar el uso de puntos de restauración dentro de una transacción.

Operaciones realizadas:

- Actualización inicial del estado de una habitación.
- Creación de un SAVEPOINT.
- Segunda actualización del estado.
- Reversión parcial mediante ROLLBACK TO SAVEPOINT.
- Confirmación de la transacción mediante COMMIT.

Comandos utilizados:

- BEGIN
- UPDATE
- SAVEPOINT
- ROLLBACK TO SAVEPOINT
- COMMIT

Resultado esperado:

Solo se conserva la primera modificación realizada antes del SAVEPOINT.

Propiedad ACID demostrada

Consistencia.

Transacción 6: Cancelar reservación

Objetivo:

Cancelar una reservación existente y liberar la habitación asociada para futuras reservaciones.

Operaciones realizadas:

- Actualización del estado de la reservación a cancelada.
- Actualización del estado de la habitación a disponible.

Comandos utilizados:

- BEGIN
- UPDATE
- COMMIT

Resultado esperado:

La reservación queda cancelada y la habitación vuelve a estar disponible para nuevos huéspedes.

Mediante el uso de los comandos BEGIN, COMMIT, ROLLBACK y SAVEPOINT es posible controlar operaciones críticas del sistema hotelero, evitando inconsistencias y asegurando el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

Además, las transacciones resultan fundamentales en procesos como reservaciones, check-in, check-out, facturación y gestión de servicios, donde múltiples operaciones deben ejecutarse como una única unidad lógica de trabajo.

Funciones

Función 1: fn_total_servicios_reservacion

Objetivo:

Calcular el monto total de los servicios adicionales consumidos durante una reservación específica.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador de la reservación que se desea consultar.

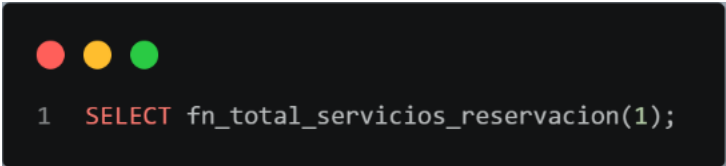
Valor de retorno:

Tipo de retorno	Descripción
numeric(10,2)	Total monetario de los servicios consumidos en la reservación.

Descripción:

La función consulta la tabla consumo_servicio y suma los subtotales asociados a la reservación indicada. Si la reservación no posee consumos registrados, retorna cero mediante la función COALESCE.

Ejemplo de uso:

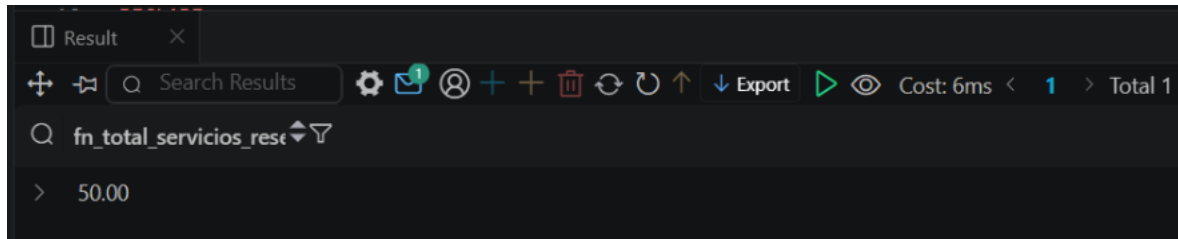
A screenshot of a terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal displays a SQL query:

```
1 SELECT fn_total_servicios_reservacion(1);
```

```
1 SELECT fn_total_servicios_reservacion(1);
```

Resultado esperado:

Devuelve el total monetario correspondiente a los servicios consumidos por la reservación indicada.



Función 2: fn_total_hospedaje_reservacion

Objetivo:

Calcular el costo total del hospedaje de una reservación.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador de la reservación que se desea calcular.

Valor de retorno:

Tipo de retorno	Descripción
numeric(10,2)	Monto total correspondiente al hospedaje.

Descripción:

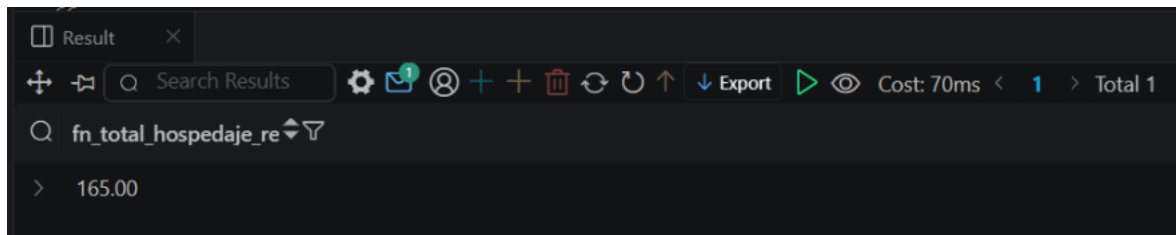
La función obtiene la cantidad de noches reservadas mediante la diferencia entre la fecha de salida y la fecha de entrada. Posteriormente multiplica dicha cantidad por el precio por noche del tipo de habitación asociado a la reservación.

Ejemplo de uso:

```
1 SELECT fn_total_hospedaje_reservacion(1);
```

Resultado esperado:

Devuelve el costo total del hospedaje correspondiente a la reservación indicada.



Función 3: fn_total_reservacion

Objetivo:

Calcular el monto total general de una reservación.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador de la reservación.

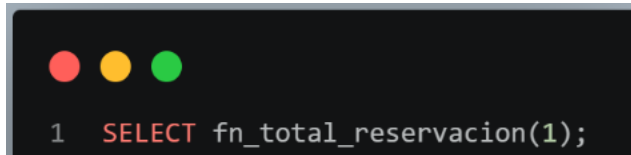
Valor de retorno:

Tipo de retorno	Descripción
numeric(10,2)	Total general de la reservación.

Descripción:

La función reutiliza las funciones `fn_total_hospedaje_reservacion` y `fn_total_servicios_reservacion` para obtener el costo total del hospedaje y el total de los servicios consumidos. Finalmente suma ambos valores para obtener el monto total de la reservación.

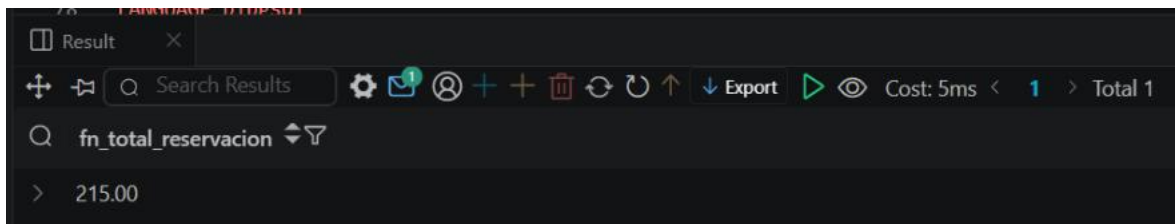
Ejemplo de uso:



```
1 SELECT fn_total_reservacion(1);
```

Resultado esperado:

Devuelve el monto total que debe pagar el huésped por concepto de hospedaje y servicios adicionales consumidos.



The screenshot shows a SQL query result in a dark-themed interface. The query is `fn_total_reservacion` and the result is `215.00`. The interface includes a search bar, a toolbar with various icons, and a status bar showing the cost as 5ms and the total as 1.

Función 4: `fn_habitaciones_por_estado`

Objetivo:

Consultar las habitaciones registradas según su estado.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
<code>p_estado</code>	<code>varchar</code>	Estado de habitación que se desea consultar.

Valor de retorno:

Campo retornado	Tipo de dato	Descripción
id_habitacion	bigint	Identificador de la habitación.
numero_habitacion	varchar	Número de habitación.
piso	integer	Piso donde se encuentra ubicada.
capacidad_maxima	integer	Capacidad máxima permitida.
estado_habitacion	varchar	Estado actual de la habitación.
tipo_habitacion	varchar	Nombre del tipo de habitación.

Descripción:

La función retorna una tabla con las habitaciones cuyo estado coincide con el parámetro recibido.

Para obtener la información completa realiza una unión entre las tablas habitacion y tipo_habitacion.

Ejemplo de uso:

```
1  SELECT *
2  FROM fn_habitaciones_por_estado('disponible');
```

Resultado esperado:

Devuelve todas las habitaciones que se encuentran en el estado especificado por el usuario.

id_habitacion	numero_habitacion	piso	capacidad_maxima	estado_habitacion	tipo_habitacion
1	101	1	1	disponible	sencilla
2	102	1	1	disponible	sencilla
4	201	2	2	disponible	doble
6	203	2	4	disponible	familiar
8	302	3	4	disponible	familiar
9	401	4	2	disponible	suite

Función 5: fn_facturacion_por_huesped

Objetivo:

Generar un reporte de facturación agrupado por huésped.

Parámetros de entrada:

Esta función no recibe parámetros.

Valor de retorno:

Campo retornado	Tipo de dato	Descripción
id_huesped	bigint	Identificador del huésped.
huesped	text	Nombre completo del huésped.
total_facturas	bigint	Cantidad de facturas asociadas.
total_facturado	numeric(10,2)	Monto total facturado al huésped.

Descripción:

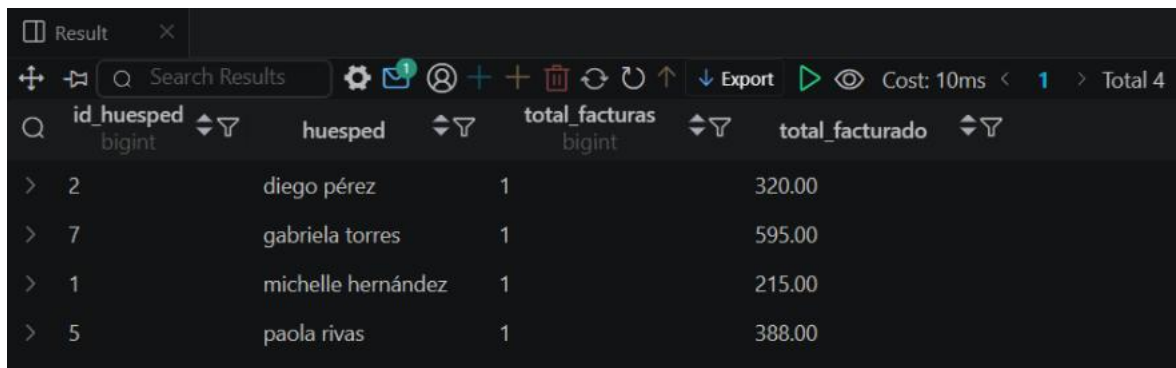
La función realiza una unión entre las tablas huesped, reservacion y factura para calcular la cantidad de facturas generadas y el total facturado por cada huésped. Los resultados son agrupados por huésped y ordenados de forma descendente según el monto total facturado.

Ejemplo de uso:

```
1 SELECT *
2 FROM fn_facturacion_por_huesped();
```

Resultado esperado:

Devuelve un reporte con los huéspedes registrados, la cantidad de facturas emitidas a cada uno y el monto total facturado.



id_huesped bigint	huesped	total_facturas bigint	total_facturado
> 2	diego p��rez	1	320.00
> 7	gabriela torres	1	595.00
> 1	michelle hern��ndez	1	215.00
> 5	paola rivas	1	388.00

Procedimientos Almacenados

Procedimiento 1: sp_registrar_reservacion

Objetivo:

Registrar una nueva reservación en el sistema, validando previamente que la habitación, el huésped y el empleado existan en la base de datos.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_fecha_entrada	date	Fecha programada de entrada del huésped.
p_fecha_salida	date	Fecha programada de salida del huésped.
p_id_habitacion	bigint	Identificador de la habitación que será reservada.
p_id_huesped	bigint	Identificador del huésped que realiza la reservación.
p_id_empleado	bigint	Identificador del empleado que registra la reservación.

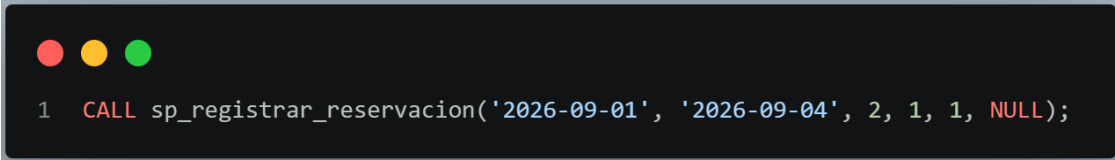
Parámetros de salida:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador generado para la nueva reservación.

Descripción:

Este procedimiento valida que la habitación, el huésped y el empleado existan antes de realizar el registro. Si alguno de estos datos no existe, se genera una excepción y la operación no se completa. Si todas las validaciones son correctas, se inserta una nueva reservación con estado confirmada y se devuelve el identificador generado.

Ejemplo de uso:



```
1 CALL sp_registrar_reservacion('2026-09-01', '2026-09-04', 2, 1, 1, NULL);
```

Resultado:

Se registra una nueva reservación y se devuelve el identificador de la reservación creada.

Procedimiento 2: sp_registrar_checkin

Objetivo:

Registrar el check-in de una reservación confirmada mediante la creación de una estancia asociada.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador de la reservación a la que se le realizará el check-in.
p_observaciones	text	Observaciones relacionadas con el ingreso del huésped.

Parámetros de salida:

Este procedimiento no devuelve parámetros de salida.

Descripción:

El procedimiento verifica que la reservación exista y que se encuentre en estado confirmada. También valida que no exista una estancia previamente registrada para la misma reservación. Si las validaciones se cumplen, se inserta un nuevo registro en la tabla estancia con la fecha y hora actual del sistema.

Ejemplo de uso:

```
1 CALL sp_registrar_checkin(3, 'Ingreso registrado por procedimiento almacenado');
```

Resultado:

Se registra el ingreso del huésped y se crea una estancia vinculada a la reservación.

Procedimiento 3: sp_registrar_checkout

Objetivo:

Registrar el check-out de una reservación, cerrar la estancia y liberar la habitación asociada.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador de la reservación a la que se le realizará el check-out.

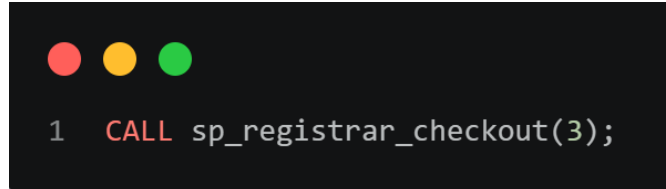
Parámetros de salida:

Este procedimiento no devuelve parámetros de salida.

Descripción:

El procedimiento verifica que exista una estancia abierta para la reservación indicada. Si existe, actualiza la fecha y hora de check-out, cambia el estado de la reservación a completada y libera la habitación asociada dejándola en estado disponible.

Ejemplo de uso:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It displays a single line of SQL code: `1 CALL sp_registrar_checkout(3);`

```
1 CALL sp_registrar_checkout(3);
```

Resultado:

La estancia queda cerrada, la reservación pasa a estado completada y la habitación queda disponible.

Procedimiento 4: `sp_registrar_consumo_servicio`

Objetivo:

Registrar el consumo de un servicio adicional realizado durante una reservación.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador de la reservación asociada al consumo.
p_id_servicio	bigint	Identificador del servicio consumido.
p_cantidad	int	Cantidad consumida del servicio.

Parámetros de salida:

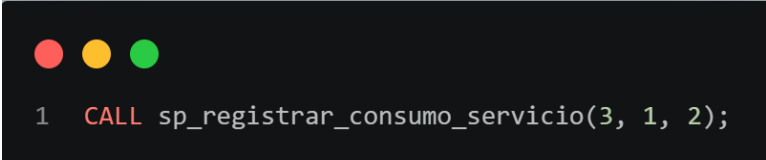
Este procedimiento no devuelve parámetros de salida.

Descripción:

El procedimiento valida que la cantidad sea mayor a cero, que el servicio exista y que la reservación indicada esté registrada. Luego obtiene el precio base del servicio e inserta el consumo

en la tabla consumo_servicio. Si existe una factura pendiente para la reservación, el trigger correspondiente recalcula automáticamente el total de la factura.

Ejemplo de uso:



```
1 CALL sp_registrar_consumo_servicio(3, 1, 2);
```

Resultado:

Se registra un nuevo consumo de servicio asociado a una reservación.

Procedimiento 5: sp_generar_factura

Objetivo:

Generar la factura de una reservación utilizando el total calculado por las funciones del sistema.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador de la reservación que será facturada.
p_metodo_pago	varchar	Método de pago seleccionado para la factura.
p_id_empleado	bigint	Identificador del empleado que genera la factura.

Parámetros de salida:

Parámetro	Tipo de dato	Descripción
p_id_factura	bigint	Identificador generado para la factura.

Descripción:

El procedimiento valida que la reservación no tenga una factura generada previamente y que el empleado exista. Luego crea la factura utilizando la función fn_total_reservacion para calcular el

total. Después inserta un detalle por hospedaje y un detalle adicional por cada servicio consumido durante la reservación.

Ejemplo de uso:



```
1 CALL sp_generar_factura(3, 'tarjeta credito', 1, NULL);
```

Resultado:

Se genera una factura para la reservación indicada junto con sus respectivos detalles de factura.

Procedimiento 6: sp_registrar_pago_factura

Objetivo:

Registrar el pago de una factura pendiente.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_factura	bigint	Identificador de la factura que será pagada.

Parámetros de salida:

Este procedimiento no devuelve parámetros de salida.

Descripción:

El procedimiento verifica que la factura exista y que su estado sea pendiente. Si la factura ya fue pagada o anulada, genera una excepción. Si la validación se cumple, actualiza el estado de la factura a pagada.

Ejemplo de uso:



```
1 CALL sp_registrar_pago_factura(1);
```

Resultado:

La factura indicada cambia su estado de pendiente a pagada.

Procedimiento 7: sp_cancelar_reservacion

Objetivo:

Cancelar una reservación existente y liberar la habitación asociada si corresponde.

Parámetros de entrada:

Parámetro	Tipo de dato	Descripción
p_id_reservacion	bigint	Identificador de la reservación que se desea cancelar.

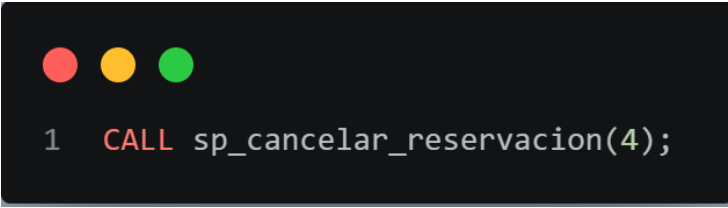
Parámetros de salida:

Este procedimiento no devuelve parámetros de salida.

Descripción:

El procedimiento obtiene el estado actual de la reservación y la habitación asociada. Si la reservación no existe, ya está cancelada o ya fue completada, genera una excepción. Si la reservación puede cancelarse, actualiza su estado a cancelada y libera la habitación si se encontraba ocupada.

Ejemplo de uso:



```
1 CALL sp_cancelar_reservacion(4);
```

Resultado:

La reservación queda cancelada y la habitación asociada queda disponible para futuras reservaciones.

Disparadores

Disparador 1: trg_reservacion_before

Objetivo:

Validar las reservaciones antes de ser registradas o modificadas, evitando conflictos de fechas sobre una misma habitación.

Tabla asociada:

reservacion

Momento de ejecución:

BEFORE INSERT OR UPDATE

Función asociada:

fn_reservacion_before_ops

Descripción:

Este disparador se ejecuta antes de insertar o actualizar una reservación. Su función principal es verificar que una habitación no se encuentre reservada en el mismo rango de fechas por otra reservación activa. También permite ajustar automáticamente el estado de la reservación cuando la fecha de salida ya pasó.

Regla de negocio relacionada:

Una habitación no puede estar reservada dos veces en el mismo período.

Disparador 2: trg_reservacion_after**Objetivo:**

Registrar auditoría sobre cambios en reservaciones y actualizar el estado de la habitación según la reservación.

Tabla asociada:

reservacion

Momento de ejecución:

AFTER INSERT OR UPDATE OR DELETE

Función asociada:

fn_reservacion_after_ops

Descripción:

Este disparador se ejecuta después de insertar, actualizar o eliminar una reservación. Registra en la tabla auditoria_reservacion los datos antes y después de la operación realizada. Además, actualiza el estado de la habitación a ocupada o disponible según el estado y las fechas de la reservación.

Regla de negocio relacionada:

Un empleado puede registrar reservaciones y gestionar el proceso de estancia del huésped.

Disparador 3: trg_validar_precio_tipo_habitacion

Objetivo:

Evitar que un tipo de habitación tenga un precio por noche inválido.

Tabla asociada:

tipo_habitacion

Momento de ejecución:

BEFORE INSERT OR UPDATE

Función asociada:

fn_validar_precio_tipo_habitacion

Descripción:

Este disparador valida que el precio por noche de un tipo de habitación sea mayor que cero antes de guardar o modificar el registro. Si el precio es menor o igual a cero, PostgreSQL rechaza la operación mediante una excepción.

Regla de negocio relacionada:

Cada tipo de habitación debe tener un precio válido para poder ser utilizado en reservaciones y facturación.

Disparador 4: trg_auditar_cambio_precio_habitacion**Objetivo:**

Registrar automáticamente los cambios realizados en el precio por noche de los tipos de habitación.

Tabla asociada:

tipo_habitacion

Momento de ejecución:

AFTER UPDATE

Función asociada:

fn_auditar_cambio_precio_habitacion

Descripción:

Este disparador se ejecuta después de modificar el precio por noche de un tipo de habitación. Registra en la tabla auditoria_precio_tipo_habitacion el precio anterior, el precio nuevo, la fecha del cambio y el usuario de base de datos que realizó la modificación.

Regla de negocio relacionada:

Los cambios en precios deben quedar registrados para control y trazabilidad administrativa.

Disparador 5: trg_evitar_borrado_servicio_esencial**Objetivo:**

Impedir la eliminación de servicios esenciales del hotel.

Tabla asociada:

servicio

Momento de ejecución:

BEFORE DELETE

Función asociada:

fn_evitar_borrado_servicio_esencial

Descripción:

Este disparador se ejecuta antes de eliminar un servicio. Su propósito es impedir que servicios esenciales como restaurante o room service sean borrados del sistema, ya que forman parte de la operación principal del hotel.

Regla de negocio relacionada:

Durante la estancia, el huésped puede consumir servicios adicionales, por lo que los servicios esenciales deben mantenerse disponibles en el sistema.

Disparador 6: trg_actualizar_total_factura_consumo**Objetivo:**

Actualizar automáticamente el total de una factura cuando se modifica el consumo de servicios.

Tabla asociada:

consumo_servicio

Momento de ejecución:

AFTER INSERT OR UPDATE OR DELETE

Función asociada:

fn_actualizar_total_factura_servicio

Descripción:

Este disparador se ejecuta después de insertar, actualizar o eliminar un consumo de servicio.

Calcula nuevamente el total de la reservación utilizando la función fn_total_reservacion y actualiza la factura correspondiente, siempre que la factura se encuentre en estado pendiente.

Regla de negocio relacionada:

La factura debe incluir el costo de la habitación y los servicios adicionales consumidos.

Conclusión

El desarrollo del sistema de gestión hotelera permitió aplicar los conocimientos adquiridos en la asignatura de Bases de Datos, desde el análisis de requisitos y reglas de negocio hasta la implementación completa de la solución en PostgreSQL. Durante el proyecto se diseñó el modelo Entidad-Relación, se realizó la transformación al modelo relacional normalizado en Tercera Forma Normal (3FN) y se implementaron las estructuras necesarias para garantizar la integridad y consistencia de los datos.

Además, se desarrollaron consultas SQL, transacciones, funciones, procedimientos almacenados y disparadores que automatizan procesos importantes como la gestión de reservaciones, estancias, servicios y facturación. Estas herramientas permiten mejorar la administración de la información y facilitan la ejecución de las operaciones del hotel.

Para finalizar, el proyecto permitió comprender la importancia de un adecuado diseño de bases de datos y demostró cómo una solución bien estructurada puede apoyar eficientemente las necesidades operativas y administrativas de una organización, en este caso de un sistema hotelero.

.

Anexos

Enlace alojado en GitHub- ER Sistema de Gestión Hotelera

Enlace alojado en GitHub- MR Sistema de Gestión Hotelera